TP Algorithmique n°3

Les boucles en Python

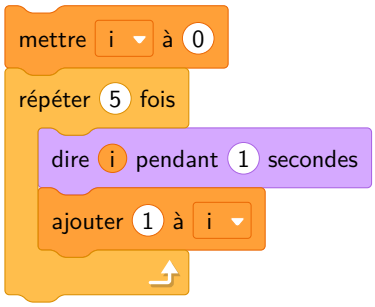
Objectifs de la séance. Faire répéter des instructions à Python : un nombre de fois connu à l'avance (boucle `for`) ou tant qu'une condition est vraie (boucle `while`) ; utiliser le reste d'une division (`%`) pour tester si un nombre est pair ou multiple d'un autre ; programmer le célèbre **FizzBuzz**.

Ouvrir Capytale depuis l'ENT et saisir le code de l'activité : .....

Partie 1 — Cours

Cours 1 : la boucle bornée `for`

En Scratch



En Python

Script

```
1 for i in range(5):
2     print(i)
```

Console

```
>>> # script executed
0
1
2
3
4
```

À retenir :

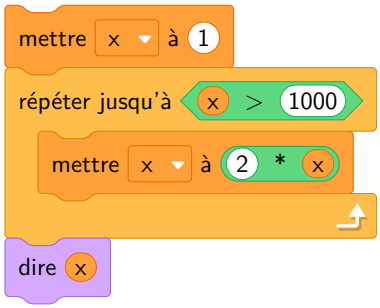
- Une boucle **bornée** répète des instructions un nombre de fois **connu à l'avance**.
- La variable `i` (le **compteur**) prend successivement toutes les valeurs de `range(...)` :

On écrit	`i` prend les valeurs
`range(5)`	0, 1, 2, 3, 4 (5 valeurs, **sans** le 5)
`range(1, 6)`	1, 2, 3, 4, 5 (de 1 à 6, **sans** le 6)
`range(2, 11, 2)`	2, 4, 6, 8, 10 (de 2 en 2)

- Comme pour `if` et `def` : deux-points `:` en fin de ligne, puis les instructions répétées sont **indentées**. Ce qui n'est plus indenté est exécuté **une seule fois**, après la boucle.

Cours 2 : la boucle non bornée `while`

En Scratch



En Python

Script

```
1 x = 1
2 while x <= 1000:
3     x = 2 * x
4 print(x)
```

Console

```
>>> # script executed
1024
```

À retenir :

- ⋆ Une boucle **non bornée** répète des instructions **tant que** (*while*) une condition est vraie ; on ne sait pas à l'avance combien de fois.
- ⋆ Attention : Scratch dit « répéter **jusqu'à** $x > 1000$ », Python dit « répéter **tant que** $x \leqslant 1000$ » : la condition est **contraire**.
- ⋆ Si la condition reste toujours vraie, la boucle ne s'arrête jamais (**boucle infinie**) ! Il faut que les instructions répétées fassent évoluer la variable testée. **Enregistrer** son travail avant d'exécuter une boucle `while`.

÷ Cours 3 : quotient et reste de la division euclidienne

La division euclidienne de 17 par 5 s'écrit $17 = 5 \times 3 + 2$: le quotient est 3 et le reste est 2.

	En Python	Résultat
Quotient	`17 // 5`	3
Reste	`17 % 5`	2

À retenir : un nombre `n` est **divisible** par `d` lorsque le reste de la division de `n` par `d` est nul :

- ⋆ `n % 2 == 0` : `n` est pair ;
- ⋆ `n % 3 == 0` : `n` est un multiple de 3.

Rappel du TP2. Pour plus de deux cas, on enchaîne `if`, `elif` (sinon si) et `else` : Python exécute **seulement le premier** bloc dont la condition est vraie.

Partie 2 — Exercices

Exercice 1 ★☆☆

Raisonner

Prévoir, puis vérifier. Pour chaque script, écrire d'abord ce qui va s'afficher, puis vérifier dans Capytale.

Script A

Script
```
1 for i in range(1, 6):
2     print(i)
```

Script B

Script
```
1 for k in range(4):
2     print(3 * k)
```

Script C

Script
```
1 for i in range(3):
2     print("Bonjour")
3 print("Fin")
```

	Je prévois	Python affiche
Script A	.....	.....
Script B	.....	.....
Script C	.....	.....

① Modifier le script A pour qu'il affiche les nombres de 1 à 20.

② Écrire un script qui affiche la table de multiplication de 7 : 7, 14, 21, . . . , 70.

.....

.....

Exercice 2 ★★☆

Raisonner

Additionner avec une boucle. Le script ci-dessous calcule une somme.

Script

```
somme = 0
for i in range(1, 11):
    somme = somme + i
print(somme)
```

① Compléter le tableau qui suit l'évolution des variables à chaque tour de boucle (on pourra s'arrêter au 5e tour).

Tour de boucle	avant	1er	2e	3e	4e	5e
Valeur de `i`	—	.....	.....	.....	.....	.....
Valeur de `somme`	0	.....	.....	.....	.....	.....

② Quelle somme ce script calcule-t-il ? Que va-t-il afficher ? Vérifier dans Capytale.

.....

.....

③ Que se passe-t-il si l'on indente la dernière ligne `print(somme)` ? Essayer.

.....

.....

④ Écrire une fonction `somme_entiers(n)` qui renvoie la somme $1 + 2 + \cdots + n$. Compléter :
`somme_entiers(10)` renvoie ..... `somme_entiers(100)` renvoie .....

.....

.....

.....

.....

Exercice 3 ★★☆

Modéliser

Placement bancaire. On place 1 000 € sur un compte rémunéré à 3 % par an : chaque année, le capital est multiplié par $CM = 1 + \dfrac{3}{100} = 1{,}03$.

① Compléter puis exécuter le script qui calcule le capital au bout de 10 ans.

Script

```
capital = 1000
for annee in range(10):
    capital = ..................
print(round(capital, 2))
```

`round(capital, 2)` arrondit au centième. Au bout de 10 ans, le capital est de ..... €.

② On cherche maintenant au bout de combien d'années le capital aura **doublé** (dépassé 2 000 €). Pourquoi une boucle `for` ne convient-elle pas ?

.....

③ Compléter le script, l'exécuter et conclure.

Script

```
capital = 1000
annee = 0
while ..............:
    capital = capital * 1.03
    annee = ............
print(annee)
```

Le capital aura doublé au bout de ..... ans.

Exercice 4 ★★☆

Chercher

Plier une feuille jusqu'au sommet de la tour Eiffel ? Une feuille de papier a une épaisseur de 0,1 mm. Chaque fois qu'on la plie en deux, son épaisseur double. La tour Eiffel mesure environ 330 m, soit 330 000 mm.

Écrire un script qui détermine le nombre de pliages nécessaires pour que l'épaisseur dépasse la hauteur de la tour Eiffel (en supposant qu'on puisse plier la feuille autant de fois qu'on veut !).

Il faut ..... pliages. L'épaisseur est alors de ..... mm, soit environ ..... m.

Exercice 5 ★☆☆

Calculer

Le reste d'une division.

① Taper chaque calcul dans la console et noter le résultat.

On tape	`17 // 5`	`17 % 5`	`20 % 4`	`2026 % 2`	`7 % 2`
Python affiche	.....	.....	.....	.....	.....

② Écrire une fonction `parite(n)` qui renvoie `"pair"` si `n` est pair, et `"impair"` sinon. La tester avec 2026 et 7.

③ À l'aide d'une boucle et de %, afficher tous les multiples de 7 compris entre 1 et 100.

Exercice 6 ★★★

Raisonner

FizzBuzz. C'est l'un des exercices de programmation les plus célèbres au monde ! On compte de 1 à 100, mais :

- à la place d'un multiple de 3, on dit `"Fizz"` ;
- à la place d'un multiple de 5, on dit `"Buzz"` ;
- à la place d'un multiple de 3 **et** de 5 (donc de 15), on dit `"FizzBuzz"`.

Le début est donc : 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz, 16, ...

① Sans ordinateur, que faut-il dire à la place de 30 ? de 33 ? de 35 ? de 49 ?

30 : ..... 33 : ..... 35 : ..... 49 : .....

② Voici une première version de la fonction `fizzbuzz(n)`. Elle contient une erreur.

Script

```
1 def fizzbuzz(n):
2     if n % 3 == 0:
3         return "Fizz"
4     elif n % 5 == 0:
5         return "Buzz"
6     elif n % 15 == 0:
7         return "FizzBuzz"
8     else:
9         return n
```

La recopier et tester `fizzbuzz(15)`. Que renvoie-t-elle ? Expliquer l'erreur, puis corriger la fonction.

.....

.....

.....

.....

③ À l'aide d'une boucle `for`, afficher le FizzBuzz complet de 1 à 100.

.....

.....

④ *Bonus.* Combien de fois dit-on `"Fizz"` entre 1 et 100 ? `"Buzz"` ? `"FizzBuzz"` ? Modifier le script pour que Python les compte.

`"Fizz"` : ..... fois `"Buzz"` : ..... fois `"FizzBuzz"` : ..... fois

.....

.....

Exercice 7 ★★★

Chercher

Pour aller plus loin : la suite de Syracuse. On choisit un entier n supérieur ou égal à 1 et on répète la règle suivante : si n est pair, on le divise par 2 ; sinon, on le multiplie par 3 et on ajoute 1. On s'arrête quand on obtient 1.

Par exemple, en partant de 6 : $6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$, soit 8 étapes.

① Écrire une fonction `syracuse(n)` qui renvoie le nombre d'étapes nécessaires pour arriver à 1. On utilisera `while`, `%` et `//`.

.....

.....

.....

.....

.....

.....

② Compléter :

`syracuse(6)` : ..... `syracuse(7)` : ..... `syracuse(27)` : .....

> Pour tous les nombres testés à ce jour, on finit toujours par arriver à 1. Mais personne n'a jamais réussi à **démontrer** que c'est vrai pour tous les entiers : c'est la **conjecture de Syracuse**, un problème ouvert depuis près de 100 ans.